

PyTorch入門

自分で書けるから、楽をしていい

rkskmt

1

前回の振り返り

- NumPy だけで多層ネットワークを組み、初回の図を再現した
- ただし **backward** は **自分で導出して書いた**

❗ クイズ

層の数や活性化関数を変えるたびに、backward を **書き直す** ことになる。100層のネットで、1ヶ所も間違えずに書ける？

- 書けるかもしれないが、**人間がやるべき仕事ではない**
- 連鎖律は機械的な作業 — 機械的な作業は機械にやらせる

2

PyTorch とは

- ディープラーニングの **業界標準ライブラリ** のひとつ
- 今日使う機能は実質2つだけ
 - **autograd** : backward を自動でやってくれる
 - **nn** : 層・損失・最適化の部品セット

i ノート

この講義の環境にはインストール済み (**environment.yml** の pytorch-cpu / torchvision)。自分のPCなら **pip install torch** 等で入る。

3

tensor : NumPy 配列の親戚

Code

```
1 import torch
2 import numpy as np
3
4 t = torch.tensor([1.0, 2.0, 3.0])
5 print(t * 2)           # 演算は NumPy とほぼ同じ感覚
6 print(t.numpy())       # NumPy 配列との相互変換も簡単
```

Result

```
tensor([2., 4., 6.])
[1.  2.  3.]
```

- 違いは2つ：**GPU に載せられる** こと、そして**勾配を覚えられる** こと

4

requires_grad : 勾配を覚える印

- まず**いちばん小さな例**で挙動を見る。 $y = x^2$ の $x = 3$ での微分は、高校の答えなら $2x = 6$

Code

```
1 import torch
2
3 x = torch.tensor(3.0, requires_grad=True)  # 「勾配を追跡せよ」の印
4 y = x ** 2
5 y.backward()                               # ← 微分はこの1行
6 print(x.grad)
```

Result

```
tensor(6.)
```

- **requires_grad=True** を付けた tensor は、**自分が関わった計算をすべて記録** する
- **backward()** を呼ぶと、記録を逆にたどって微分が計算され、**x.grad** に入る
- x^2 で当たるのは当たり前？ 一次で、**前回1往復ぶん手計算したネットワーク** に同じことをやらせる

5

山場：autograd が手計算を言い当てる

- バックプロパゲーションの回と **まったく同じ** ネットワーク・同じ数値を流す

```
Code
1 a = torch.tensor([0.3, -0.2], requires_grad=True) # α1, α2
2 b = torch.tensor(0.1, requires_grad=True)
3 w = torch.tensor(0.5, requires_grad=True)
4 c = torch.tensor(-0.1, requires_grad=True)
5 x = torch.tensor([1.0, 0.5]) # 入力（正解 y=1）
6
7 z = a @ x + b # アフィン変換
8 h = torch.relu(z) # ReLU
9 u = w * h + c # 出力層
10 p = torch.sigmoid(u) # シグモイド
11 L = -torch.log(p) # 交差エントロピー（正解1）
12
13 L.backward() # ← 復路はこの1行
14
15 print(f"L = {L.item():.4f}")
16 print(f"∂L/∂α = {a.grad.numpy().round(4)}")
17 print(f"∂L/∂b = {b.grad.item():.4f}")
18 print(f"∂L/∂w = {w.grad.item():.4f}")
19 print(f"∂L/∂c = {c.grad.item():.4f}")
```

```
Result
L = 0.6685
∂L/∂α = [-0.2438 -0.1219]
∂L/∂b = -0.2438
∂L/∂w = -0.1463
∂L/∂c = -0.4875
```

① 答え合わせ

手計算（とNumPy検算）の答えは α : $(-0.2438, -0.1219)$ 、 b : -0.2438 、 w : -0.1463 、 c : -0.4875 。 **全桁一致**。 `loss.backward()` の中身は、前回自分の手でやった連鎖律の機械化にすぎない。

なぜ自動でできるのか

- forward の間、PyTorch は **計算の履歴（計算グラフ）** を記録している
- `backward()` は、その履歴を **逆順にたどって連鎖律を適用** するだけ
- やっていることは [前回の手計算1往復](#) と同じ：
 - 往路の途中の値を保存（だからメモリを食う）
 - δ を出力側から掛けて貯める

💡 ヒント

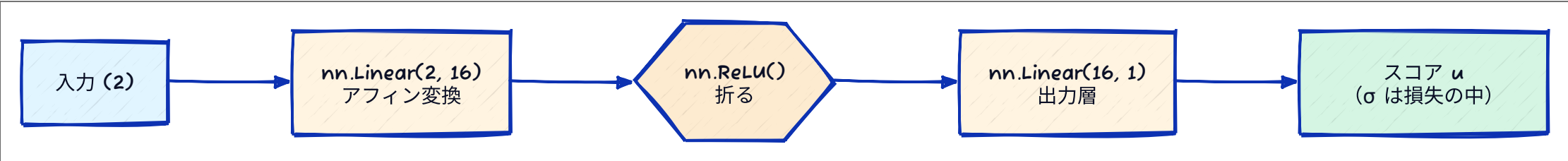
ブラックボックスとして使うのと、中身を知って使うのは別物。 われわれは **中身を手で再現済み** なので、堂々と楽をしてよい。

nn：部品には全部、見覚えがある

```
Code
1 import torch.nn as nn
2
3 model = nn.Sequential(
4     nn.Linear(2, 16),    # アフィン変換  $X @ W1 + b1$  (パラメータは自動で用意・初期化)
5     nn.ReLU(),          # 折る
6     nn.Linear(16, 1),    # 出力層  $H @ w2 + c2$ 
7 )
8 print(model)
```

Result

```
Sequential(
  (0): Linear(in_features=2, out_features=16, bias=True)
  (1): ReLU()
  (2): Linear(in_features=16, out_features=1, bias=True)
)
```



PyTorch の名前	この講義で作った部品
nn.Linear	アフィン変換 ($X @ W1 + b1$)
nn.ReLU	$\max(0, z)$
nn.BCEWithLogitsLoss	シグモイド＋交差エントロピー (打ち消しもやってくれる)
torch.optim.SGD	勾配降下法 ($\theta -= lr * grad$)
loss.backward()	バックプロパゲーション

- 新しい概念はゼロ。全部、名前が付いただけ

nn.Linear の中身を覗く

- 部品を1個だけ取り出して、素性を確認する

Code

```
1 import torch
2 import torch.nn as nn
3
4 torch.manual_seed(0)
5 lin = nn.Linear(2, 3)          # 2次元 → 3個の変換
6 print(lin.weight)             # W (3行2列) ランダム初期化済み
7 print(lin.bias)               # b (3個)
8
9 x = torch.tensor([[1.0, 2.0]])
10 print(lin(x))                 # Linear に通す
11 print(x @ lin.weight.T + lin.bias)  # 自分で計算しても、同じ
```

Result

```
Parameter containing:
tensor([[ -0.0053,  0.3793],
        [-0.5820, -0.5204],
        [-0.2723,  0.1896]], requires_grad=True)
Parameter containing:
tensor([-0.0140,  0.5607, -0.0628], requires_grad=True)
tensor([[ 0.7393, -1.0621,  0.0441]], grad_fn=<AddmmBackward0>)
tensor([[ 0.7393, -1.0621,  0.0441]], grad_fn=<AddBackward0>)
```

- 中身は本当に **ただのアフィン変換** — 重みの持ち方が転置 (**W.T**) なことだけ流儀が違う
- **requires_grad=True** も最初から付いている (**nn** の部品は学習される前提)

学習ループ：3拍子

Code

```
1 from sklearn.datasets import make_moons
2
3 X, y = make_moons(n_samples=200, noise=0.1, random_state=42)
4 X_t = torch.tensor(X, dtype=torch.float32)
5 y_t = torch.tensor(y, dtype=torch.float32).unsqueeze(1) # (200,) → (200,1)
6
7 torch.manual_seed(0)
8 model = nn.Sequential(nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 1))
9 loss_fn = nn.BCEWithLogitsLoss()
10 optimizer = torch.optim.SGD(model.parameters(), lr=0.5)
11
12 for i in range(1, 4001):
13     optimizer.zero_grad() # ① 前回の勾配をリセット
14     logits = model(X_t)
15     loss = loss_fn(logits, y_t)
16     loss.backward() # ② backprop (自動)
17     optimizer.step() # ③ 勾配降下で1歩
18
19     if i in (1, 100, 500, 2000, 4000):
20         acc = ((torch.sigmoid(logits) >= 0.5).float() == y_t).float().mean()
21         print(f"更新 {i:5d}回 | Loss: {loss.item():.4f} | 正答率: {acc.item():.1%}")
```

Result

更新	1回		Loss: 0.6807		正答率: 50.0%
更新	100回		Loss: 0.2652		正答率: 85.5%
更新	500回		Loss: 0.0449		正答率: 99.0%
更新	2000回		Loss: 0.0057		正答率: 100.0%
更新	4000回		Loss: 0.0024		正答率: 100.0%

- NumPy 版と同じく **100%**。 **zero_grad → backward → step** の3拍子はどんなネットでも同じ

3拍子の①は、なぜ要る？

❗ クイズ

`optimizer.zero_grad()` を消したら、何が起きると思う？

Code

```
1 import torch
2
3 theta = torch.tensor(2.0, requires_grad=True)
4 opt = torch.optim.SGD([theta], lr=0.1)
5
6 loss = theta ** 2
7 loss.backward()
8 print(theta.grad)    # dL/dθ = 2θ = 4
9 opt.step()
10 print(theta)         # 2 - 0.1 × 4 = 1.6 (step は勾配降下そのもの)
11
12 loss = theta ** 2    # zero_grad しなくて、もう1周
13 loss.backward()
14 print(theta.grad)    # 2θ = 3.2 のはず.....が？
```

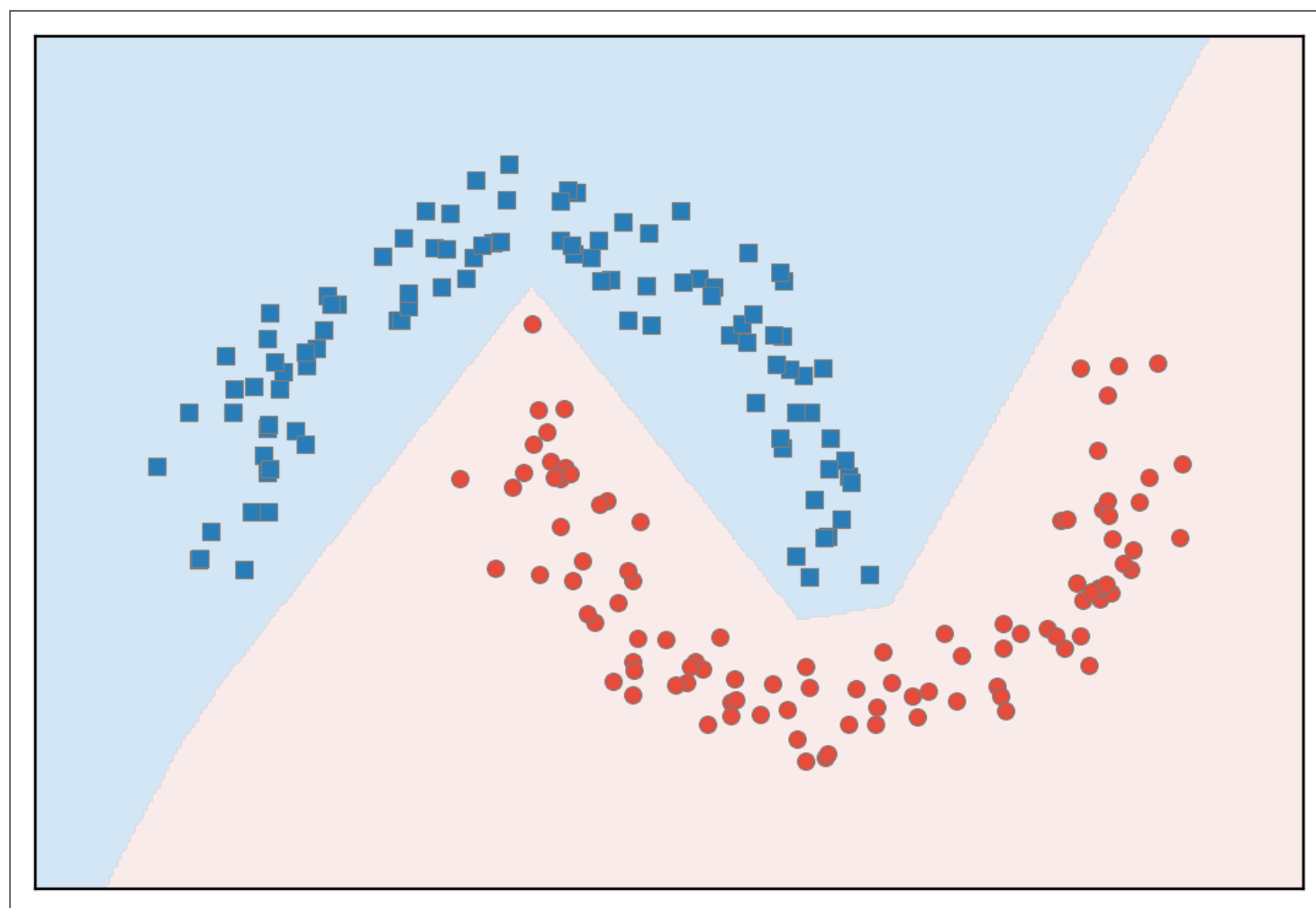
Result

```
tensor(4.)
tensor(1.6000, requires_grad=True)
tensor(7.2000)
```

- **grad** は上書きではなく **足し込み** ($4 + 3.2 = 7.2$) — 前回の勾配が残ったまま歩くと方向を誤る
- だから毎周リセットが①。 **step()** が③ ($\theta -= lr * grad$ そのもの)

境界も同じものが出る

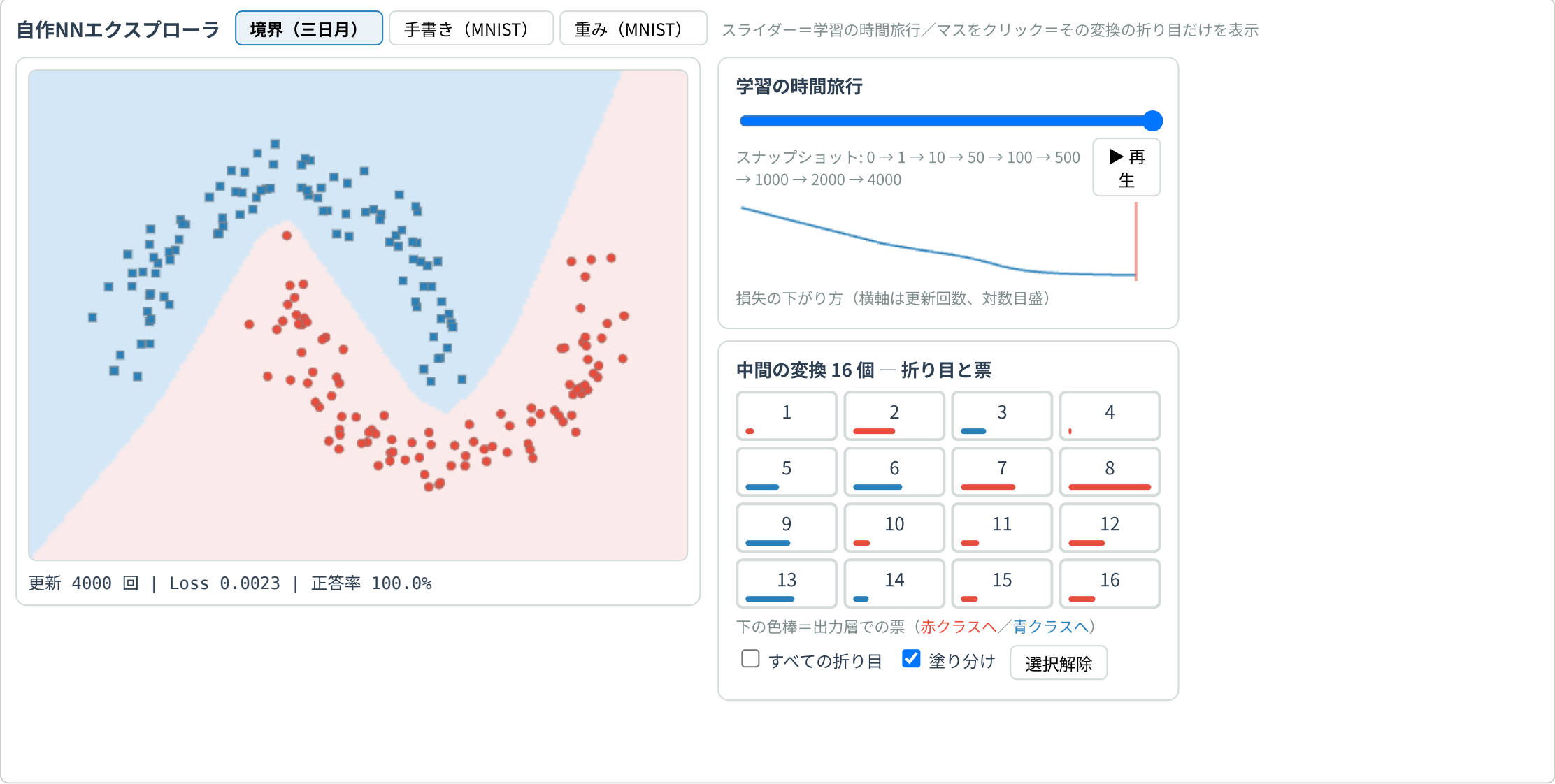
▶ 描画コード



- NumPy 版で書いた forward / backward 計60行ほどが、PyTorch では**学習ループ込みで十数行**
- 浮いた労力は、**モデルとデータを考えること**に使う

学習後の16人の票を読む

学習が終わった中間の変換16個は、それぞれ**どちらのクラスに票を投じる係**になったのか — 実物で確認：



エクスプローラを別タブで開く

- マスの下の色棒＝出力層の重み（票）。**赤に押す係10人・青に押す係6人**に自然と分業している
- 票の大きさは係ごとにまるで違う（最大 6.05、ほぼ無票の 0.22 も） — 無票の係の折り目はどこに立っている？ クリックして確かめる
- この分業、誰も指示していない。損失と勾配降下だけでこうなった



- 学習ループのコード を書き換えて実行（境界の図まで続けて実行すると変化が見える）

1

中間の変換を2個に減らす

```
nn.Linear(2, 2), nn.ReLU(),  
nn.Linear(2, 1)
```

2

ReLU を素通しに変える

```
nn.ReLU() → nn.Identity()
```

3

学習率を下げる

```
lr=0.05
```

💡 ここで観察

- 変換2個：「変換を重ねる」の回では2個でも分けられたのに、ここでは**87%止まり**（`torch.manual_seed(3)`だと50%まで転落）。表現力はあっても、**良い谷にたどり着けるかは別問題** — 16個の「余裕」は学習の保険でもある（ローカルミニマムの罠）
- `nn.Identity()`：折り目ゼロ — **境界が直線に戻り、87%で頭打ち**。「線形は重ねても線形」を PyTorch でも再確認
- どれを変えても、学習ループの3拍子は**一切書き換えない**で済む — それが部品化の威力

14

まとめ

- **autograd**：`loss.backward()` = 連鎖律の自動化。手計算と全桁一致を確認した
- **nn の部品**は全部、この講義で自作した部品の別名（対応表）
- 学習ループは **zero_grad → backward → step** の3拍子
- 中身を知っているから、安心して楽ができる

① 次回予告：最終回

道具は完成した。次回、初回に約束した**手書き数字を読み取る機械**を作る。入力は 28×28 ピクセル — **784次元**。次元の話、覚えてる？

15

練習問題

以下の問題を **Pythonコード** で解け。

問題1：autograd に自分の手計算を言い当てさせる

バックプロパゲーションの回の**練習問題1** ($x = (0.5, 1.0)$ 、 $y = 0$ 、 $\alpha = (0.4, 0.3)$ 、 $b = -0.1$ 、 $w = -0.5$ 、 $c = 0.2$) を PyTorch で組み、**loss.backward()** の勾配が自分の手計算と一致することを確認めよ。

- 1. 本編の「山場」のコードを、上の数値に差し替える（正解が $y = 0$ なので損失は $L = -\log(1 - p)$ ）
- 2. 5つの勾配を表示し、手計算の答え $(-0.125, -0.25)$ 、 -0.25 、 0.2 、 0.5 と照合せよ

問題2：ReLU に折られた入力でも試す

入力を $x = (-1.0, 0.5)$ に変えて実行し、 α, b, w の勾配が**ちょうどゼロ**になることを確かめよ。（バックプロパゲーションの回の練習問題2 — autograd も、折られた経路には勾配を流さない）

16

解答例

▶ 解答例を見る

```
Result
|-----|
| x = (0.5, 1.0) | L = 0.6931
|   ∂L/∂α = [-0.125 -0.25 ], ∂L/∂b = -0.2500, ∂L/∂w = 0.2000, ∂L/∂c = 0.5000
| x = (-1.0, 0.5) | L = 0.7981
|   ∂L/∂α = [-0.   0. ], ∂L/∂b = 0.0000, ∂L/∂w = 0.0000, ∂L/∂c = 0.5498
|-----|
```

- 問題1：**全桁一致**。autograd は連鎖律の機械化であって、魔法ではない
- 問題2： **$\text{ReLU}'(z) = 0$** の「ゼロで潰す」まで、忠実に再現される

17